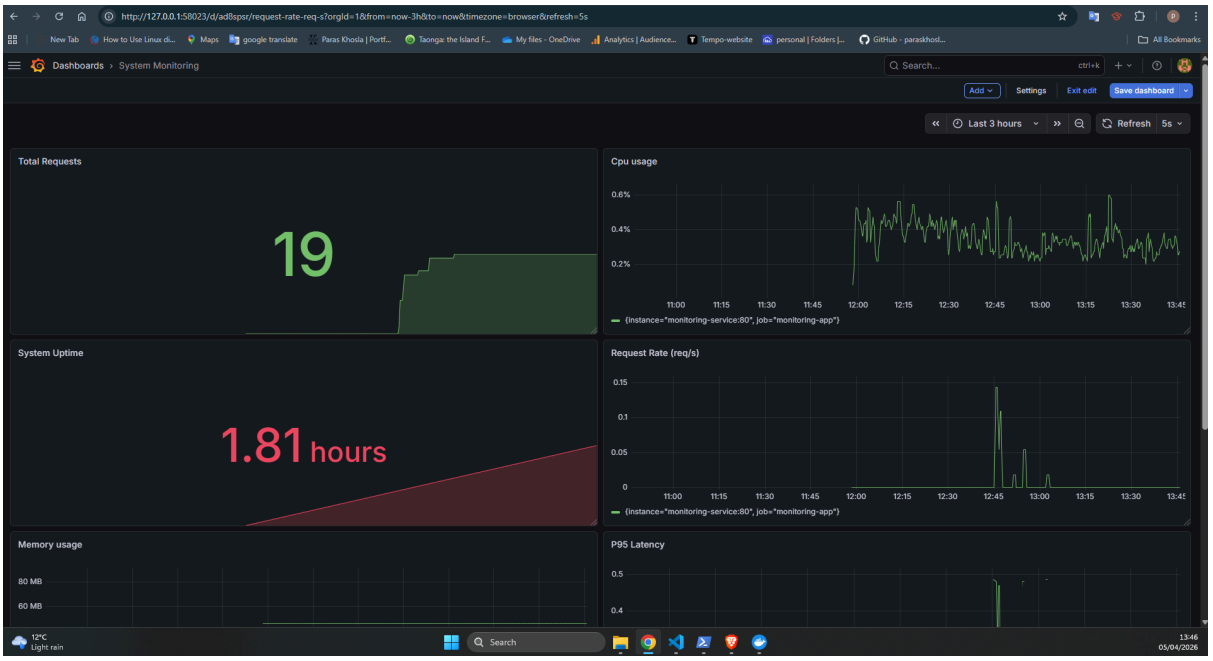
Cloud-Native Monitoring System using Kubernetes, Prometheus & Grafana



Author: Paras khosla
Date: 10-12-25

Table of Content

1. Introduction 3
2. Problem Statement 3
3. Solution Overview 3
4. Architecture 3
5. Key Features 3
6. Dashboard Insights 4
7. Challenges 4
Conclusion 4
Future Improvements 4
References 5

1. Introduction

This project implements a cloud-native monitoring system for a containerized application using Kubernetes, Prometheus, and Grafana. The system provides real-time observability into application performance, including request rate, error rate, latency, and system resource usage. It simulates a production-like environment and demonstrates how modern monitoring tools can be integrated to build an end-to-end observability pipeline.

2. Problem Statement

Modern distributed applications require continuous monitoring to ensure reliability and performance. Without proper observability, issues such as latency spikes, system failures, and resource bottlenecks can remain undetected. The goal of this project is to design and implement a monitoring solution that provides real-time insights into application behavior and system health.

3. Solution Overview

The solution consists of a FastAPI-based application instrumented with Prometheus metrics, containerized using Docker, and deployed on a Kubernetes cluster (Minikube). Prometheus is responsible for collecting and storing metrics, while Grafana is used to visualize these metrics through interactive dashboards. This setup reflects a typical cloud-native monitoring architecture used in modern production systems.

4. Architecture

This architecture demonstrates how application-level metrics are generated, collected, and visualized in a distributed environment. Each component plays a specific role in ensuring observability across the system.

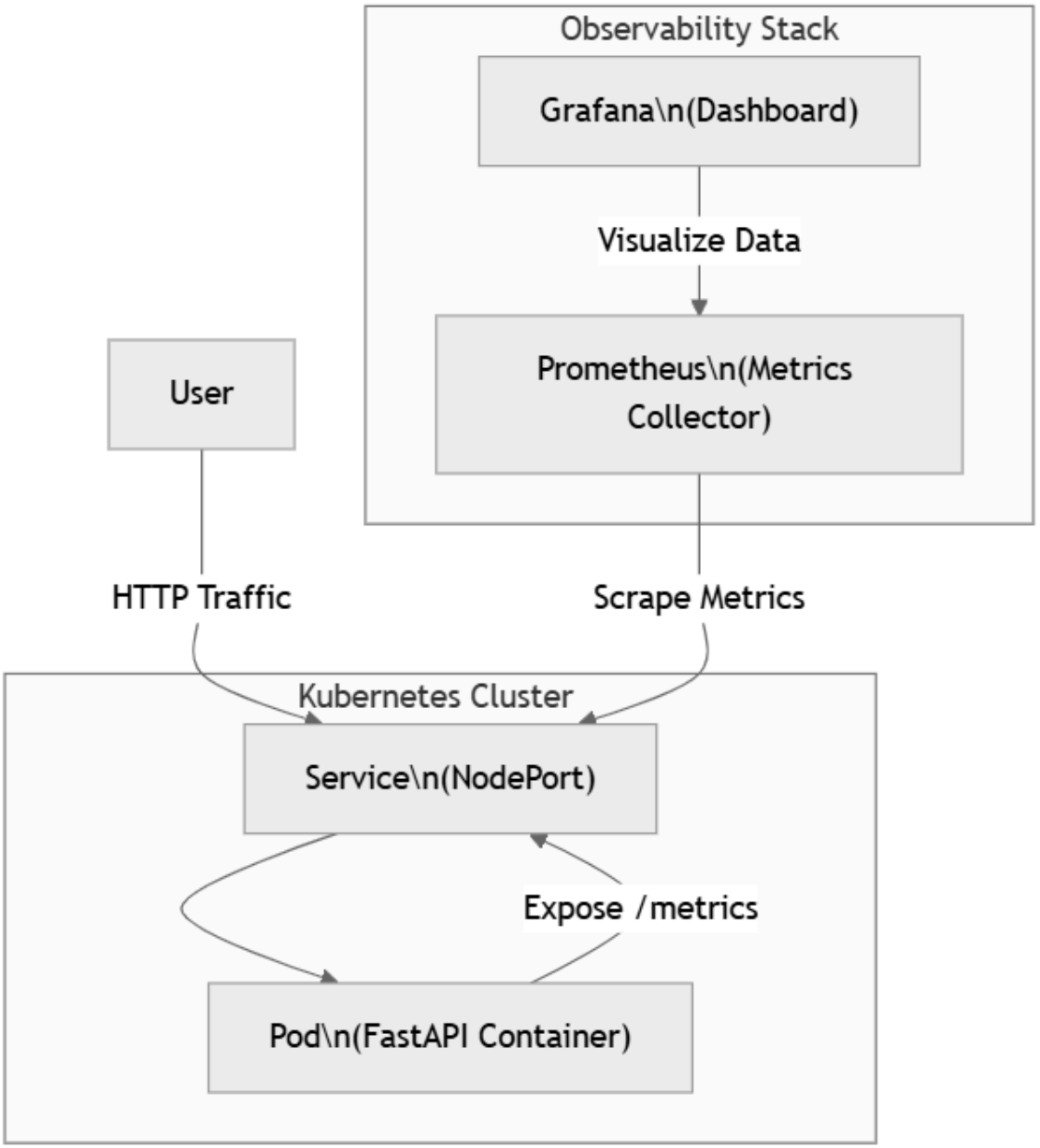


Figure 1: System Architecture of the Monitoring Pipeline

5. Key Features

- Real-time monitoring of requests and errors
- Latency tracking using histogram metrics
- CPU and memory usage monitoring
- Containerized deployment using Docker
- Kubernetes-based orchestration
- Interactive dashboards using Grafana

6. Dashboard Insights

The Grafana dashboard provides insights into system behavior and performance:

- Request rate reflects system load
- Error rate helps identify failures
- Latency metrics highlight performance bottlenecks
- CPU and memory usage indicate system health

These insights enable proactive monitoring and faster issue detection.

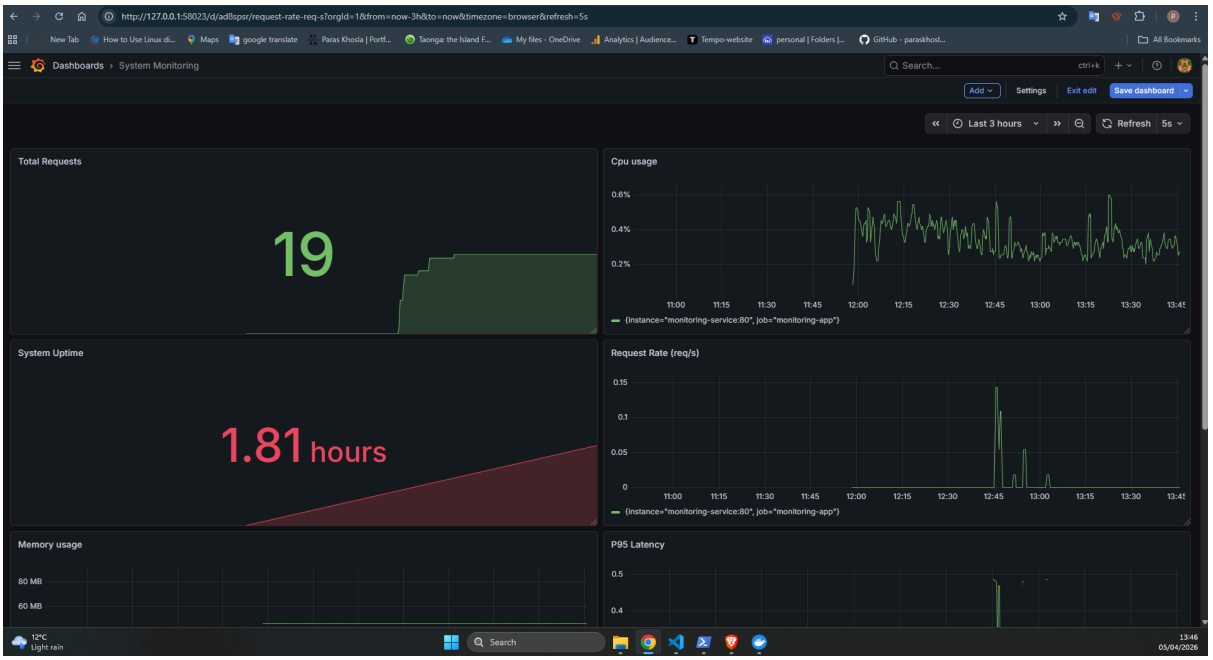


Figure 2: Grafana Dashboard showing system metrics

7. Challenges

Problem:
Prometheus was unable to scrape metrics from the application.

Cause:
The `/metrics` endpoint returned JSON instead of the required Prometheus format.

Solution:
The endpoint was updated to return metrics in `text/plain` format.

```
    return { data : "Here is your data" }

# @app.get("/metrics")
# def metrics():
#     return generate_latest()

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

Figure 3: Fix applied to return Prometheus-compatible format

Result:
After fixing the issue, Prometheus successfully scrapped the metrics, and the target status changed to "UP".

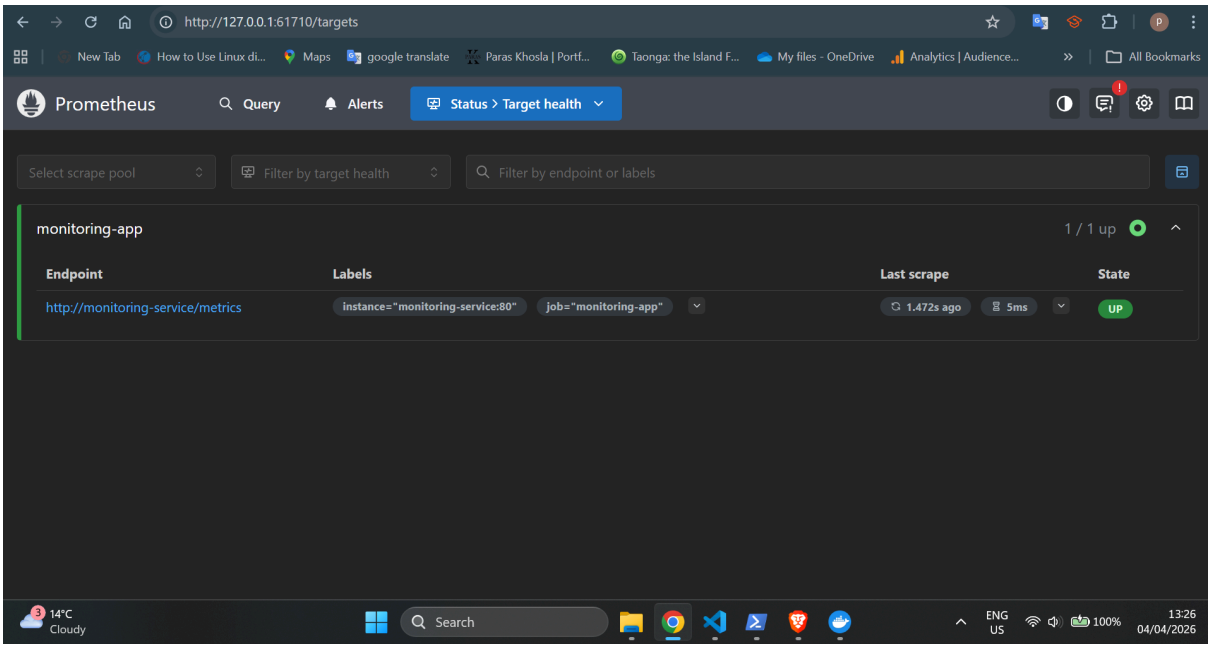


Figure 4: Prometheus successfully scraping metrics

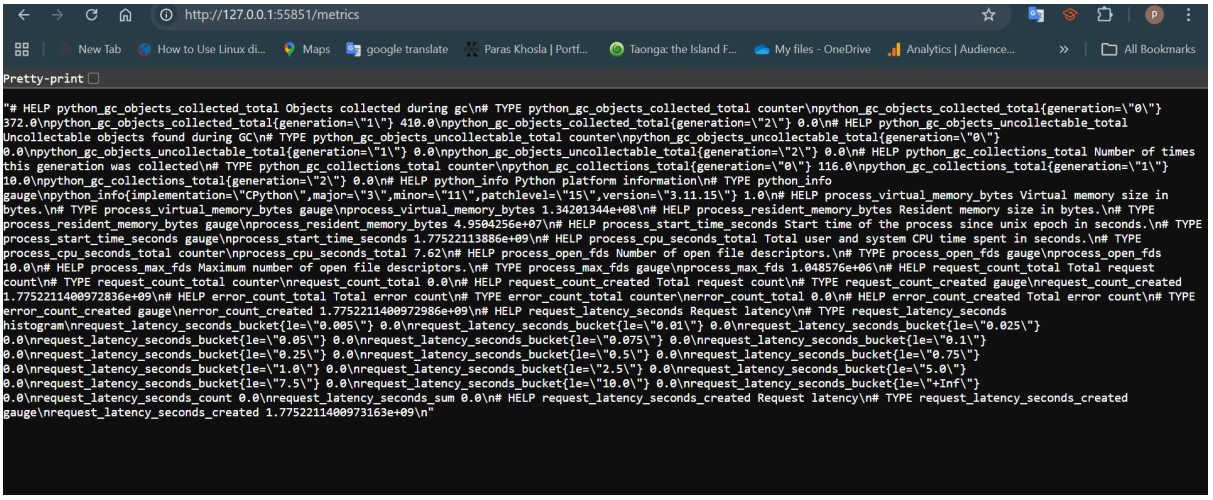


Figure 5: Prometheus metrics exposed via /metrics endpoint

Conclusion

This project demonstrates the implementation of a complete cloud-native monitoring pipeline using industry-standard tools. It highlights the importance of observability in modern systems and provides practical experience in integrating application instrumentation, metric collection, and data visualization. The project also provided hands-on experience in debugging real-world monitoring issues, reinforcing the importance of correct configurations and data formats in distributed systems.

Future Improvements

- Implement alerting using Prometheus Alertmanager
- Deploy the system on a cloud platform (Azure AKS or AWS EKS)
- Use Helm charts for deployment automation
- Add persistent storage for Grafana dashboards

References

Prometheus Authors. (n.d.). Prometheus documentation. Retrieved April 2026, from https://prometheus.io/docs/introduction/overview/

Grafana Labs. (n.d.). Grafana documentation. Retrieved April 2026, from https://grafana.com/docs/grafana/latest/

The Kubernetes Authors. (n.d.). Kubernetes documentation. Retrieved April 2026, from https://kubernetes.io/docs/home/

FastAPI Developers. (n.d.). FastAPI documentation. Retrieved April 2026, from https://fastapi.tiangolo.com/